SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06 **COURSE NAME: MACHINE LEARNING**

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – Data Frame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting

Name: K. Dedeepya

Reg no: 192324252

1. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

Problem Understanding

A massive numerical dataset must be processed using NumPy under limited memory conditions. The main constraints are:

1. Mathematical operations must be performed efficiently.
2. Random sampling should not create unnecessary copies.
3. Statistical calculations should use minimum memory.
4. Array creation, indexing, slicing, and aggregation must be optimized.
5. Memory overflow and performance bottlenecks must be identified and debugged.

Proposed Solution

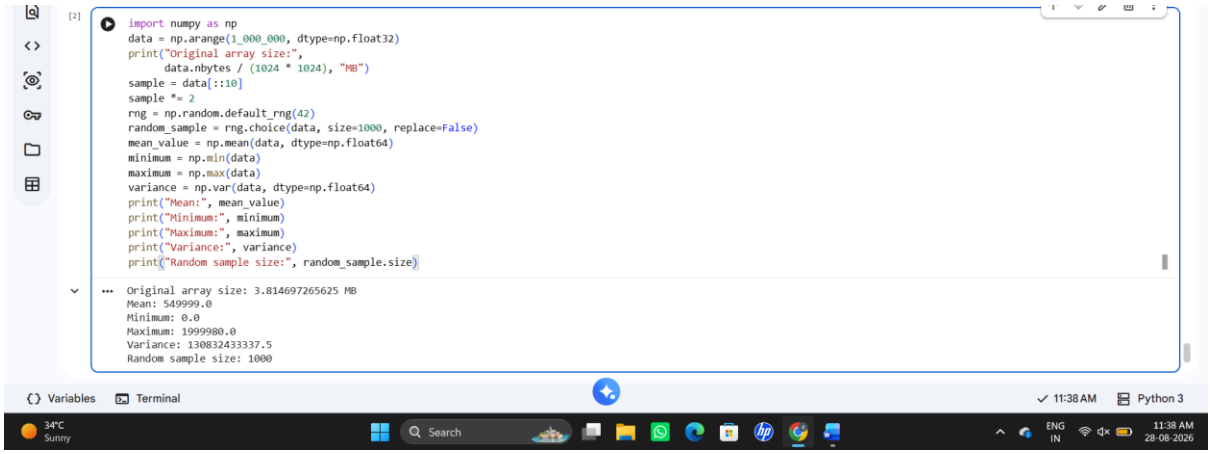
NumPy arrays are suitable because they store numerical data efficiently in contiguous memory and support vectorized operations. Instead of creating multiple temporary arrays, the dataset should be processed using appropriate data types, slicing, views, and in-place operations.

Techniques Used

Technique	Purpose
np.empty()	Creates an array without unnecessary initialization
dtype	Controls memory usage
Slicing	Processes only required portions of an array
Views	Access data without creating a copy

In-place operations	Reduce temporary memory allocation
Vectorization	Performs operations without Python loops
np.random.choice()	Performs random sampling
np.mean(), np.sum()	Efficient aggregation
np.min(), np.max()	Statistical analysis

NumPy Program



Memory Optimization

Instead of using:

data = **np. arrange (1_000_000, dtype=np. float64)**

we can use:

data = **np. arrange (1_000_000, dtype=np. float32)**

A float32 value requires approximately 4 bytes, while float64 requires approximately 8 bytes. Therefore, float32 can significantly reduce memory usage when high precision is not required.

Slicing such as: sample = data[::10]

is also efficient because NumPy can create a view of the original array instead of copying the complete dataset.

Indexing and Aggregation Optimization

Indexing should be restricted to the required portion of the dataset.

subset = data [10000:20000]

Boolean filtering can also be used: filtered = data [data > 500000]

For large datasets, unnecessary copies should be avoided. Aggregation functions such as sum (), mean (), min (), and max () should be performed directly on the required array or slice.

Debugging Strategies

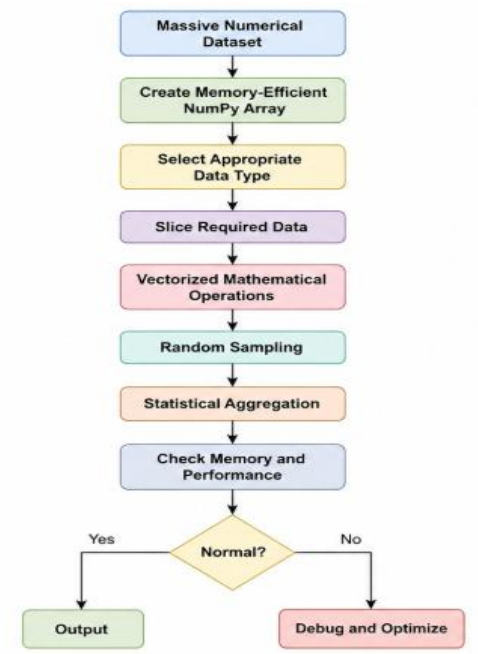
If memory overflow occurs:

1. Check the array size using data. N bytes.
2. Use smaller numerical data types such as float32.
3. Process data in chunks.
4. Avoid unnecessary copies.
5. Use slicing instead of copying.
6. Perform operations in-place where possible.

For performance problems:

1. Avoid Python for loops.
2. Use NumPy vectorized operations.
3. Check unnecessary conversions.
4. Profile the program to identify slow operations.
5. Reduce repeated calculations.

Flowchart



Conclusion

The memory constraint can be satisfied by using efficient NumPy data types, slicing, views, vectorization, random sampling, and in-place operations. Aggregation should be performed only on required data. Memory usage and execution time should be continuously monitored to achieve efficient large-scale numerical processing.

2. You are given a large dataset of financial transactions containing duplicate records, inconsistent timestamps, and invalid entries. Design a data cleaning pipeline using NumPy and Pandas with constraints: (i) remove duplicates without losing valid data, (ii)

standardize time formats, and (iii) filter out invalid transactions. Explain how indexing, grouping, and sorting ensure data integrity.

Problem Understanding

A large financial transaction dataset may contain duplicate records, inconsistent timestamps, and invalid transactions. The cleaning pipeline must satisfy the following constraints:

1. Remove duplicate records without deleting valid transactions.
2. Standardize all timestamp formats.
3. Filter invalid transactions.
4. Preserve valid financial information.
5. Maintain data integrity using indexing, grouping, and sorting.

Proposed Data Cleaning Pipeline

The following stages are used:

Raw Financial Data

↓

Load CSV using Pandas

↓

Inspect Data Types and Missing Values

↓

Remove Exact Duplicates

↓

Standardize Timestamps

↓

Validate Transaction Values

↓

Filter Invalid Records

↓

Group and Check Transaction IDs

↓

Reset Index

↓

Clean Dataset

Program

```
print("\nCleaned Dataset:")
print(df)
print("\nFinal number of records:", len(df))
```

```
Original records: 20
Duplicate transaction IDs:
Empty DataFrame
Columns: [transaction_id, timestamp, amount, currency]
Index: []

Cleaned Dataset:
   transaction_id           timestamp  amount currency
0            T001 2023-01-01 10:00:00   10.50      USD
1            T002 2023-01-01 10:05:00   20.00      EUR
2            T003 2023-01-01 10:10:00    5.00      USD
3            T006 2023-01-02 11:05:00   25.00      EUR
4            T007 2023-01-02 11:10:00   12.75      USD
5            T008 2023-01-03 12:00:00    8.99      USD
6            T009 2023-01-03 12:05:00   30.00      EUR
7            T011 2023-01-04 13:00:00   15.00      USD
8            T012 2023-01-04 13:10:00   10.00      USD
9            T013 2023-01-05 14:00:00    5.00      EUR
10           T014 2023-01-05 14:05:00   18.00      USD
11           T015 2023-01-05 14:10:00    7.50      USD
12           T016 2023-01-06 15:00:00    2.00      EUR
13           T017 2023-01-06 15:05:00   11.25      USD
14           T018 2023-01-06 15:10:00    9.00      EUR

Final number of records: 15
```

Use of Indexing

Indexing allows specific records or columns to be accessed efficiently.

df.loc[df["amount"] > 10000]

This selects transactions whose amount is greater than 10,000.

The index is reset after cleaning: df.reset_index(drop=True)

This ensures that the final DataFrame has a consistent sequential index.

Use of Filtering

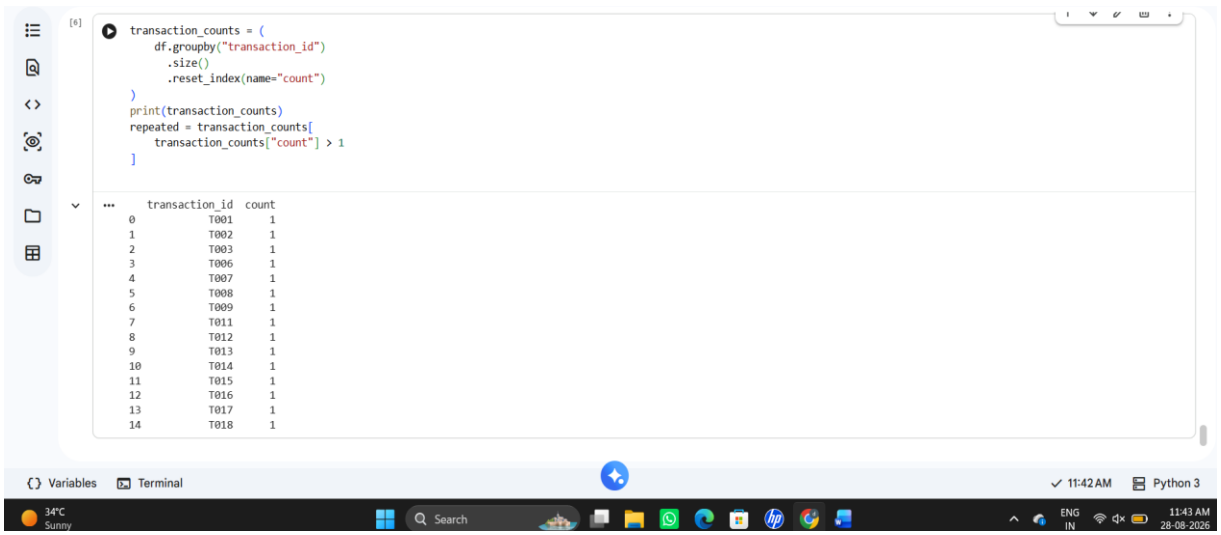
Filtering removes invalid transactions.

df = df[df["amount"] > 0]

Transactions having zero or negative values are removed because they do not satisfy the defined transaction constraint.

Use of Grouping

Grouping can be used to identify suspicious or repeated transaction IDs.



```
transaction_counts = (
    df.groupby("transaction_id")
      .size()
      .reset_index(name="count")
)
print(transaction_counts)
repeated = transaction_counts[
    transaction_counts["count"] > 1
]
```

```
   transaction_id  count
0            T001      1
1            T002      1
2            T003      1
3            T006      1
4            T007      1
5            T008      1
6            T009      1
7            T011      1
8            T012      1
9            T013      1
10           T014      1
11           T015      1
12           T016      1
13           T017      1
14           T018      1
```

Use of Sorting

Transactions are sorted chronologically: **df = df.sort_values("timestamp")**

This is important for financial analysis because transaction order can affect fraud detection, balance calculations, and time-based analysis.

Data Integrity Table

Constraint	Pandas/NumPy Technique	Purpose
Remove duplicates	drop_duplicates()	Removes exact duplicate records
Standardize timestamps	pd.to_datetime()	Converts timestamps into a common format
Validate amount	pd.to_numeric()	Converts amount into numerical form
Remove invalid amount	Boolean filtering	Removes zero/negative values
Check repeated IDs	groupby()	Detects repeated transactions
Maintain order	sort_values()	Creates chronological order
Consistent indexing	reset_index()	Recreates sequential index

Debugging Strategy

If unexpected duplicates remain:

1. Check whether the duplicate is an exact duplicate.
2. Compare transaction IDs.
3. Group records using group by().
4. Check timestamp differences.
5. Verify whether repeated IDs represent legitimate transactions.

If timestamps cannot be converted:

```
invalid_dates = df[pd.to_datetime(df["timestamp"],
    errors="coerce").isna()]
print(invalid_dates)
```

This identifies records containing invalid timestamp values.

Conclusion

The financial transaction dataset can be cleaned safely using Pandas duplicate removal, timestamp conversion, numerical validation, filtering, grouping, sorting, and indexing. The pipeline removes invalid data while preserving valid transactions and provides debugging mechanisms for identifying data-quality problems.

3. A dataset contains millions of records with mixed data types and must be transformed into a structured format suitable for machine learning. Constraints include: (i) efficient type conversion, (ii) handling missing and categorical data, and (iii) ensuring consistent indexing. Explain how you will use Panda's operations and NumPy functions to preprocess data while maintaining computational efficiency.

Problem Understanding

A dataset containing millions of records may include numerical, categorical, textual, and missing values. Before using it for machine learning, the data must be converted into a structured and consistent format.

The major constraints are:

1. Efficient type conversion.
2. Handling missing values.
3. Processing categorical variables.
4. Maintaining consistent indexing.
5. Minimizing computational and memory overhead.

Proposed Preprocessing Pipeline

Raw Large Dataset

↓

Load Data Using Pandas

↓

Inspect Data Types

↓

Convert Numerical Columns

↓

Handle Missing Values

↓

Encode Categorical Data

↓

Remove Invalid Records

↓

Convert to NumPy Arrays

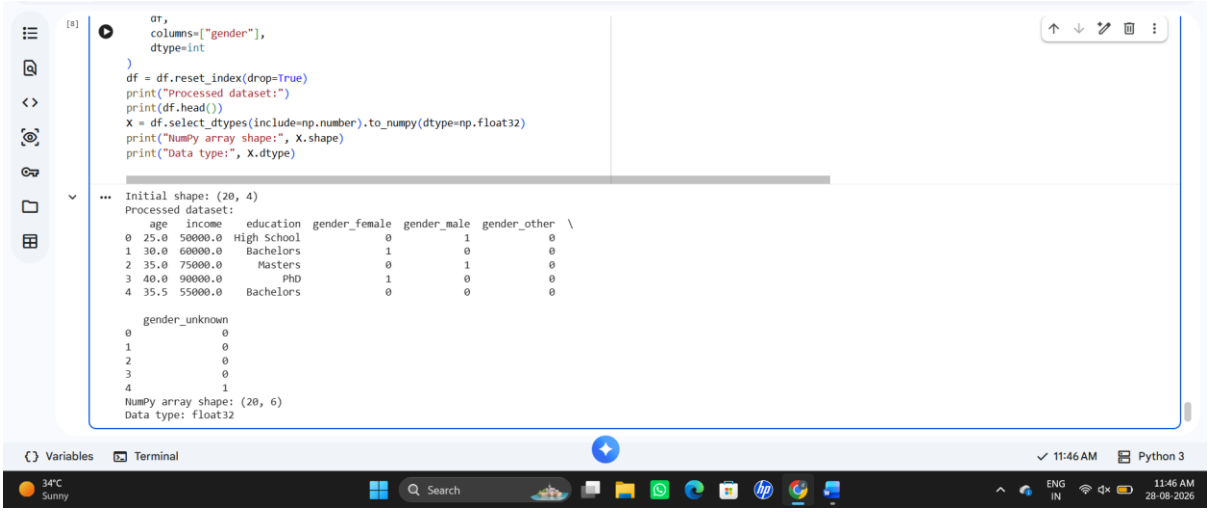
↓

Reset Index

↓

Machine Learning Ready Dataset

Program



Efficient Type Conversion

Type conversion should be performed only on required columns.

df["age"] = pd.to_numeric(df["age"], errors="coerce")

The errors="coerce" option converts invalid numerical values into NaN, which can then be handled systematically.

For large numerical datasets, float32 can be used when the required numerical precision permits it.

X = df.to_numpy (dtype=np. float32)

Handling Missing Data

Missing numerical values can be replaced using statistical values such as the median.

df["income"] = df["income"].fillna(

df["income"].median())

Median is useful because it is less affected by extreme values than the mean.

For categorical values: **df["gender"] = df["gender"].fillna("unknown")**

Handling Categorical Data

Categorical values should be standardized before encoding.

For example:

Male

male

MALE

can be converted into:

male

using:

df["gender"] = (df["gender"].astype("string").str.strip().str.lower())

Categorical values can then be converted into numerical representations using one-hot encoding.

Indexing and Filtering

Specific columns can be selected using:

features = df[["age", "income"]]

Filtering can remove invalid records:

df = df[df["age"] > 0]

Multiple conditions can also be applied:

df = df[(df["age"] > 0) & (df["income"] >= 0)]

Efficiency Techniques

Operation	Efficient Technique
Large CSV loading	read_csv() with suitable options
Numerical conversion	pd.to_numeric()
Missing values	fillna()
Categorical memory reduction	category dtype
Filtering	Boolean indexing
Numerical processing	NumPy vectorization
ML conversion	to_numpy()
Index consistency	reset_index()

Debugging

The following checks can be used:

print(df.info())

print(df.isnull().sum())

print(df.dtypes)

print(df.shape)

To detect duplicate records:

print(df.duplicated().sum())

To detect invalid ages:

print(df[df["age"] <= 0])

To check categorical values:

print(df["gender"].value_counts())

These checks help identify missing values, incorrect types, invalid values, and inconsistent categories before model training.

Conclusion

Pandas provides efficient tools for handling large mixed-type datasets, while NumPy provides fast numerical processing. Type conversion, missing-value handling, categorical encoding, filtering, indexing, and vectorized operations transform raw data into a consistent machine-learning-ready structure while maintaining computational efficiency.

4. A financial analytics system must compute real-time risk metrics from streaming stock market data under strict latency constraints. Using NumPy, design a solution that performs vectorized computations for returns, volatility, and correlations. Ensure minimal memory usage and fast execution. Explain how slicing, broadcasting, and aggregation functions are optimized, and discuss debugging techniques for incorrect calculations or delayed outputs.

Problem Understanding

A financial analytics system receives streaming stock market data and must calculate risk metrics with very low latency. The required calculations include:

1. Stock returns.
2. Volatility.
3. Correlations.
4. Vectorized processing.
5. Low memory usage.
6. Fast execution.

Proposed Solution

NumPy is appropriate because it performs vectorized numerical operations using optimized array computations. Instead of processing every stock price using Python loops, complete arrays can be processed simultaneously.

Processing Flowchart

Streaming Stock Prices

↓

Store Recent Price Window

↓

NumPy Array

↓

Slice Required Prices

↓

Calculate Returns

↓

Calculate Volatility

↓

Calculate Correlation

↓

Aggregate Risk Metrics

↓

Real-Time Risk Output

Program

Returns Calculation

The percentage return is calculated as:

Return = (Current Price - Previous Price) / Previous Price

Using NumPy:

returns = (prices[:, 1:] - prices[:, :-1]) / prices[:, :-1]

This operation is vectorized and calculates returns for all stocks without explicit Python loops.

Use of Slicing

The following slicing operations obtain adjacent prices:

prices[:, 1:]

prices[:, :-1]

prices[:, 1:] represents current prices, while prices[:, :-1] represents previous prices.

This avoids manually accessing each individual element.

Broadcasting

NumPy broadcasting allows arithmetic operations between arrays of compatible dimensions without explicitly creating repeated arrays.

For example:

returns = (prices[:, 1:] - prices[:, :-1]) / prices[:, :-1]

NumPy automatically performs the operations element by element.

Volatility

Volatility measures the variation in returns.

volatility = np.std(returns, axis=1)

The axis=1 parameter calculates volatility independently for each stock.

Correlation

Correlation measures how the returns of different stocks move relative to each other.

correlation_matrix = np.corrcoef(returns)

A value close to:

+1 → Strong positive relationship

0 → Weak/no linear relationship

-1 → Strong negative relationship

Memory Optimization

For real-time processing, only a recent window of prices needs to be retained.

For example: **recent_prices = prices[:, -100:]**

This prevents unnecessary storage of the complete historical dataset.

Additional optimization techniques include:

Technique	**Benefit**
NumPy arrays	Compact numerical storage
Slicing	Processes required data only
Vectorization	Reduces Python-loop overhead

Broadcasting	Avoids explicit repeated arrays
axis aggregation	Efficient stock-wise calculations
Rolling window	Limits memory requirements
Appropriate dtype	Controls numerical memory usage

Debugging Incorrect Calculations

Check the dimensions:

print(prices.shape)

print(returns.shape)

Check for invalid values:

print(np.isnan(prices).sum())

print(np.isinf(prices).sum())

Check whether prices are positive:

print(np.any(prices <= 0))

Manually verify one return:

manual_return = (prices[0, 1] - prices[0, 0]) / prices[0, 0]

print(manual_return)

Debugging Delayed Outputs

If processing becomes slow:

1. Reduce the historical window.
2. Avoid Python loops.
3. Use vectorized NumPy operations.
4. Avoid unnecessary array copies.
5. Calculate only required metrics.
6. Monitor array dimensions and memory usage.

Conclusion

NumPy provides an efficient solution for real-time financial risk analysis. Vectorized returns, broadcasting, slicing, standard deviation, and correlation operations minimize computation time. A limited rolling data window reduces memory usage, making the system suitable for low-latency financial analytics.

5. You are required to integrate and analyse data from multiple IoT devices with varying sampling rates and missing timestamps. Constraints include: (i) aligning time- series data correctly, (ii) handling missing intervals without distortion, and (iii) maintaining

computational efficiency. Explain how you will use Panda's time-series functions, reindexing, interpolation, and aggregation to enforce these constraints, and discuss debugging techniques for misaligned or inconsistent time-series data.

Problem Understanding

IoT systems may contain multiple devices that collect data at different sampling rates. Some devices may also have missing timestamps. The main constraints are:

1. Align time-series data correctly.
2. Handle missing time intervals without introducing distortion.
3. Maintain computational efficiency.
4. Use reindexing and interpolation appropriately.
5. Validate the final time-series dataset.

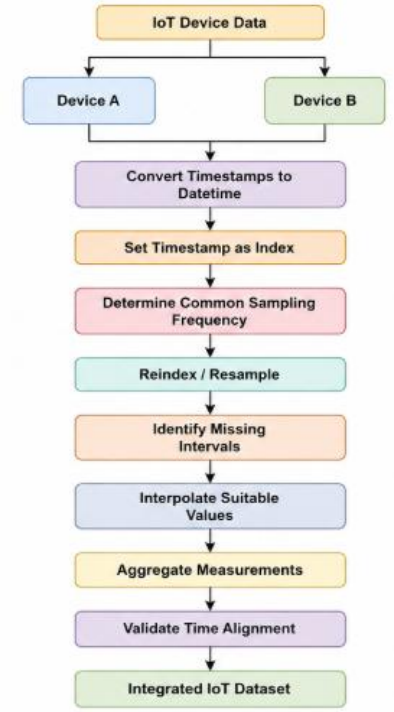
Proposed Solution

Pandas provides specialized time-series operations such as:

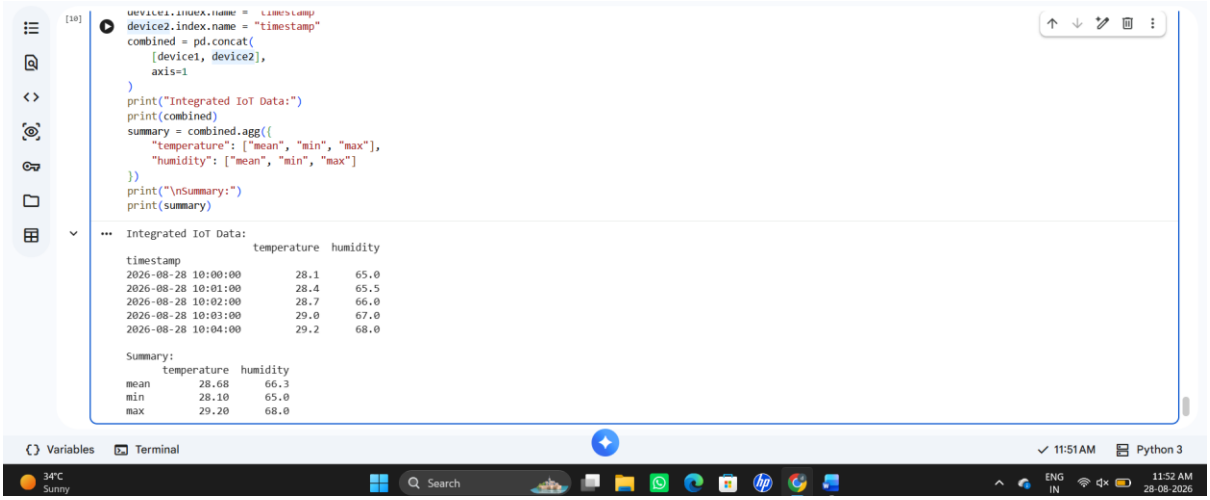
- to_datetime()
- set_index()
- resample()
- reindex()
- interpolate()
- groupby()
- rolling()
- merge_asof()

These functions can be used to synchronize IoT measurements.

Processing Flowchart



Program



```
device2.index.name = "timestamp"
combined = pd.concat(
    [device1, device2],
    axis=1
)
print("Integrated IoT Data:")
print(combined)
summary = combined.agg({
    "temperature": ["mean", "min", "max"],
    "humidity": ["mean", "min", "max"]
})
print("\nSummary:")
print(summary)
```

```
Integrated IoT Data:
                     temperature  humidity
timestamp
2026-08-28 10:00:00         28.1      65.0
2026-08-28 10:01:00         28.4      65.5
2026-08-28 10:02:00         28.7      66.0
2026-08-28 10:03:00         29.0      67.0
2026-08-28 10:04:00         29.2      68.0

Summary:
      temperature  humidity
mean        28.68      66.3
min         28.10      65.0
max         29.20      68.0
```

Timestamp Alignment

First, timestamps are converted into Pandas datetime objects.

df["timestamp"] = pd.to_datetime(df["timestamp"])

This ensures that all devices use a consistent time representation.

The timestamp is then used as the DataFrame index:

df = df.set_index("timestamp")

Time-based indexing makes it easier to perform resampling, reindexing, and interpolation.

Reindexing

A common time index is created:

common_index = pd.date_range(start="2026-08-28 10:00", end="2026-08-28 10:04", freq="min")

Each device is then reindexed against the same timeline.

device1 = device1.reindex(common_index)

device2 = device2.reindex(common_index)

This ensures that measurements from different devices correspond to the same time intervals.

Handling Missing Intervals

Reindexing may introduce NaN values when a device did not record a measurement at a particular time.

For example:

10:00 → 28.1

10:01 → 28.4

10:02 → NaN

10:03 → 29.0

For suitable continuous sensor measurements, time-based interpolation can estimate the missing value:

df["temperature"] = (df["temperature"]. interpolate(method="time"))

Interpolation should only be used when it is logically appropriate. For events or discrete measurements, filling a missing value by interpolation may create incorrect information.

Aggregation

Pandas aggregation can calculate summary statistics:

combined.agg({

"temperature": ["mean", "min", "max"],

"humidity": ["mean", "min", "max"]})

For larger time-series datasets, resampling can be used:

hourly = combined.resample("h").mean()

This converts high-frequency readings into hourly averages.

Constraint Validation

Constraint	Technique	Purpose
Timestamp consistency	pd.to_datetime()	Standardizes timestamps
Time alignment	reindex()	Creates common timeline

Missing intervals	isna()	Detects missing values
Missing numerical values	interpolate()	Estimates suitable missing values
Different sampling rates	resample()	Converts to common frequency
Data combination	concat() / merge()	Integrates devices
Statistical analysis	agg()	Generates summaries
Efficient access	Datetime index	Enables fast time-based operations

Debugging Misaligned Data

Check the timestamp index:

print(device1.index)

print(device2.index)

Check missing values:

print(combined.isna().sum())

Check duplicated timestamps:

print(combined.index.duplicated().sum())

Check the sampling interval:

print(combined.index.to_series().diff())

Check the final data range:

print(combined.index.min())

print(combined.index.max())

If timestamps are not aligned, verify:

1. Timestamp format.
2. Time zone information.
3. Sampling frequency.
4. Duplicate timestamps.
5. Missing timestamps.
6. Incorrect start and end times.

Important Data Integrity Rule

Interpolation should not automatically be applied to every missing value. If a sensor was disconnected for a long period, interpolation may produce unrealistic readings. Therefore, the system should define a maximum acceptable gap before interpolation.

For example:

df["temperature"] = (df["temperature"]. Interpolate (method="time", limit=2))

This limits interpolation to short gaps.

Conclusion

Panda's time-series functionality provides an efficient solution for integrating IoT devices with different sampling rates and missing timestamps. Converting timestamps, setting a datetime index, reindexing, resampling, interpolating appropriate short gaps, and aggregating measurements ensures correct temporal alignment while maintaining computational efficiency and data integrity.